



BITS Pilani

Processor Architectures and Design

• Evaluation



Evaluation Component	Name	Type	Weight	Duration	Day, Date, Session, Time
EC-1	Quiz1	Online	5%	7 days	TBD
	Situated learning Assignment – I	Online	10%	21 days	TBD
	Lab Quiz1	Online	5%	7 days	TBD
	Lab Assignment -I	Online	10%	21 days	TBD
EC - 2	Mid-Semester Test	Closed Book	30%	2 hours	September 19-20, 2026
EC - 3	Comprehensive Exam	Open Book	40%	2 ½ hours	December 05-06, 2026



Introduction to Parallel Processing

- Parallel processing means performing multiple operations at the same time to improve speed, throughput, or energy efficiency. Modern processors use parallelism because simply increasing clock frequency is no longer enough due to power, heat, and memory limits.
- Example: A web server may handle many user requests in parallel; a GPU may process thousands of pixels simultaneously; a CPU may execute multiple independent instructions at once.

- RISC & Chip transistor count accelerate the growth curve
- Transistor count facilitates translation of x86
- Since 2003 power and Instruction Level Parallelism(ILP) limits negatively impacted growth
- The transition from ILP to:
 - data-level parallelism (DLP)
 - thread-level parallelism (TLP)
 - request-level parallelism (RLP)
- Example: ILP: adding two registers while another instruction loads memory; DLP: adding two arrays element-by-element; TLP: two threads running on two cores.

Why Processor Architects Need Parallelism



Performance growth today depends on exploiting hardware concurrency rather than only faster transistors. The course description connects this need to scalar pipelines, multicore processors, GPUs, vector architecture, and AI-oriented processors.

Example: A mobile SoC may use a CPU core for control logic, a GPU for graphics, and an AI accelerator for image recognition.

Classes of Computers



- Personal mobile devices (PMD): cost, energy, media performance, responsiveness
- Desktop: price-performance, energy, graphics perf
- Server: throughput, availability, scalability, energy
- Warehouse: price-performance, throughput, energy
- Embedded: price, energy application-targeted performance

Feature	Personal mobile device (PMD)	Desktop	Server	Clusters/warehouse-scale computer	Internet of things/ embedded
Price of system	\$100–\$1000	\$300–\$2500	\$5000–\$10,000,000	\$100,000–\$200,000,000	\$10–\$100,000
Price of microprocessor	\$10–\$100	\$50–\$500	\$200–\$2000	\$50–\$250	\$0.01–\$100
Critical system design issues	Cost, energy, media performance, responsiveness	Price-performance, energy, graphics performance	Throughput, availability, scalability, energy	Price-performance, throughput, energy proportionality	Price, energy, application-specific performance

Figure 1.2 A summary of the five mainstream computing classes and their system characteristics. Sales in 2015 included about 1.6 billion PMDs (90% cell phones), 275 million desktop PCs, and 15 million servers. The total number of embedded processors sold was nearly 19 billion. In total, 14.8 billion ARM-technology-based chips were shipped in 2015. Note the wide range in system price for servers and embedded systems, which go from USB keys to network routers. For servers, this range arises from the need for very large-scale multiprocessor systems for high-end transaction processing.

Parallelism: Application/Architecture



- Application: data-/task-level parallelism (DLP, TLP)
- Architecture:
 1. Instruction-level parallelism: exploits data-level parallelism at modest levels with compiler help using ideas like pipelining and at medium levels using ideas like speculative execution.
 2. Vector architectures, graphic processor units (GPUs), and multimedia instruction sets exploit data-level parallelism by applying a single instruction to a collection of data in parallel.

- 3. Thread-level parallelism exploits either data-level parallelism or task-level parallelism in a tightly coupled hardware model that allows for interaction between parallel threads.
- 4. Request-level parallelism exploits parallelism among largely decoupled tasks specified by the programmer or the operating system.

Computer categories



- 1. Single instruction stream, single data stream (SISD)—
This category is the uniprocessor. The programmer thinks of it as the standard sequential computer, but it can exploit ILP. Ex. superscalar and speculative execution.
- 2. Single instruction stream, multiple data streams (SIMD)— exploit data-level parallelism by applying the same operations to multiple items of data in parallel. Each processor has its own data memory (hence, the MD of SIMD), but there is a single instruction memory and control processor, which fetches and dispatches instructions. Ex. vector architectures, multimedia extensions to standard instruction sets, and GPUs.

- 3. Multiple instruction streams, single data stream (MISD)—No commercial multiprocessor of this type has been built to date
- 4. Multiple instruction streams, multiple data streams (MIMD)—In general, MIMD is more flexible than SIMD and thus more generally applicable, but it is inherently more expensive than SIMD. For example, MIMD computers can also exploit data-level parallelism, although the overhead is likely higher than would be seen in a SIMD computer. This overhead means that grain size must be sufficiently large to exploit the parallelism efficiently.



Instruction Set Architecture: The Myopic View of Computer Architecture



- The contract between software and hardware
 - Multiple machines can implement ISA
 - Advantage – program portability
 - Microprocessor design – starts with ISA
-
- ISA produces – microarchitecture
 - Microarchitecture has to be rigorously verified.

- Class of ISA: general register, load/store, stack, and others
- Register memory address ISA: 80x86
- Load store ISA: ARMv8 and RISC-V
- All ISAs announced since 1985 are load-store.
- Memory addressing: byte/word addressing(80x86, ARMv8, and RISC-V); aligned(ARMv8)

- Addressing modes: Addressing mode indicates a way of locating data or operands.
- Types and size of operands: operand sizes of 8-bit (ASCII character), 16-bit (Unicode character or half word), 32-bit (integer or word), 64-bit (double word or long integer), and IEEE 754 floating point in 32-bit (single precision) and 64-bit (double precision).
- Operations: data transfer, arithmetic logical, control, and floating point.
- Control flow instructions: conditional branches, unconditional jumps, procedure calls, and returns.
- Encoding an ISA: fixed or variable length

Arch, Implementation & Realization



Architecture

- ISA
- Functional level behavior of processor

Implementation

- Micro-architecture
- Logic structure that implements the arch

Realization

- Physical Implementation

- Architecture: Covering all aspects of computer design—instruction set architecture, organization (or micro-architecture), and hardware.
- Overall the role of the architect is to design a computer (or system) to meet functional requirements and performance
- requirements where performance includes consideration of all significant requirements including price, power, etc.

Functional requirements	Typical features required or supported
<i>Application area</i>	<i>Target of computer</i>
Personal mobile device	Real-time performance for a range of tasks, including interactive performance for graphics, video, and audio; energy efficiency (Chapters 2–5 and 7; Appendix A)
General-purpose desktop	Balanced performance for a range of tasks, including interactive performance for graphics, video, and audio (Chapters 2–5; Appendix A)
Servers	Support for databases and transaction processing; enhancements for reliability and availability; support for scalability (Chapters 2, 5, and 7; Appendices A, D, and F)
Clusters/warehouse-scale computers	Throughput performance for many independent tasks; error correction for memory; energy proportionality (Chapters 2, 6, and 7; Appendix F)
Internet of things/embedded computing	Often requires special support for graphics or video (or other application-specific extension); power limitations and power control may be required; real-time constraints (Chapters 2, 3, 5, and 7; Appendices A and E)
<i>Level of software compatibility</i>	<i>Determines amount of existing software for computer</i>
At programming language	Most flexible for designer; need new compiler (Chapters 3, 5, and 7; Appendix A)
Object code or binary compatible	Instruction set architecture is completely defined—little flexibility—but no investment needed in software or porting programs (Appendix A)
<i>Operating system requirements</i>	<i>Necessary features to support chosen OS (Chapter 2; Appendix B)</i>
Size of address space	Very important feature (Chapter 2); may limit applications
Memory management	Required for modern OS; may be paged or segmented (Chapter 2)
Protection	Different OS and application needs: page versus segment; virtual machines (Chapter 2)
<i>Standards</i>	<i>Certain standards may be required by marketplace</i>
Floating point	Format and arithmetic: IEEE 754 standard (Appendix J), special arithmetic for graphics or signal processing
I/O interfaces	For I/O devices: Serial ATA, Serial Attached SCSI, PCI Express (Appendices D and F)
Operating systems	UNIX, Windows, Linux, CISCO IOS
Networks	Support required for different networks: Ethernet, Infiniband (Appendix F)
Programming languages	Languages (ANSI C, C++, Java, Fortran) affect instruction set (Appendix A)

Trends



- Integrated circuit logic technology: transistor density, memory, disk, and network.
- Semiconductor DRAM (dynamic random-access memory)—the foundation of main memory. The growth of DRAM has slowed dramatically, from quadrupling every three years as in the past.
- Semiconductor Flash (electrically erasable programmable read-only memory)—nonvolatile semiconductor memory, the standard storage device in PMDs. In recent years, the capacity per Flash chip increased by about 50%–60% per year, doubling roughly every 2 years. Currently, Flash memory is 8–10 times cheaper per bit than DRAM.

Trends



- Magnetic disk technology—One way to increase disk capacity is to add more platters at the same areal density, but there are already seven platters within the one-inch depth of the 3.5-inch form factor disks. There is room for at most one or two more platters.
- Heat Assisted Magnetic Recording(HAMR) is the last chance for continued improvement in areal density of hard disk drives, which are now 8–10 times cheaper per bit than Flash and 200–300 times cheaper per bit than DRAM. This technology is central to server- and warehouse-scale storage.
- Network technology—Network performance depends both on the performance of switches and on the performance of the transmission system.

Trends



- Bandwidth and latency: trade latency for bandwidth often works
- Power and Energy:
- Dynamic Voltage-Frequency Scaling (DVFS)
- Design for the typical case
- Overclocking/turbo mode
- Cost

Bandwidth and latency



- Latency (execution time): time to finish a fixed task
- Throughput (bandwidth): number of tasks in fixed time
- Different: exploit parallelism for throughput, not latency (e.g., bread)
- Often contradictory (latency vs. throughput)

Measuring and Reporting Performance



- Execution time:
- CPU time = Instruction count * Clock cycles per instruction * Clock cycle time
- “X is n times as fast as Y” will mean
- Execution time_Y/Execution time_X=n
- Since execution time is the reciprocal of performance, the following relationship holds:

$$n = \frac{\text{Execution time}_Y}{\text{Execution time}_X} = \frac{\frac{1}{\text{Performance}_Y}}{\frac{1}{\text{Performance}_X}} = \frac{\text{Performance}_X}{\text{Performance}_Y}$$

Amdahl's Law:

$$\text{Speedup}_{\text{overall}} = \frac{\text{Execution time}_{\text{old}}}{\text{Execution time}_{\text{new}}}$$
$$= \frac{1}{(1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}}}$$

 Deeply pipelined machines

 Many instructions/cycle

 Out-of-order execution of instructions

 Aggressive branch prediction techniques

Exmple: Computers A and B implement the same ISA. Computer A has a clock cycle time of 250 ps and an effective CPI of 2.0 for some program and computer B has a clock cycle time of 500 ps and an effective CPI of 1.2 for the same program. Which computer is faster and by how much?

- Assume each computer executes I instructions,
- so CPU time $A = I \times 2.0 \times 250 = 500 \times I$ ps
- CPU time $B = I \times 1.2 \times 500 = 600 \times I$ ps
- A is faster by the ratio of execution times

$$\frac{\text{performance}_A}{\text{performance}_B} = \frac{\text{execution_time}_B}{\text{execution_time}_A} = \frac{600 \times I}{500 \times I} = 1.2$$

Serial Computation



- Traditionally, software has been written for *serial* computation:
 - To be run on a single computer having a single Central Processing Unit (CPU)
 - A problem is broken into a discrete series of instructions
 - Instructions are executed one after another
 - Only one instruction may execute at any moment in time

Parallel Computing



- In the simplest sense - *parallel computing* is the simultaneous use of multiple computing resources to solve a computational problem:
 - To be run using multiple CPUs
 - A problem is broken into discrete parts that can be solved concurrently
 - Each part is further broken down into a series of instructions
 - Instructions from each part execute simultaneously on different CPUs

- The computational problem should be able to:
 - Be broken apart into discrete pieces of work that can be solved simultaneously
 - Execute multiple program instructions at any moment in time
 - Be solved in less time with multiple compute resources than with a single compute resource.

Functional Parallelism

Instruction level

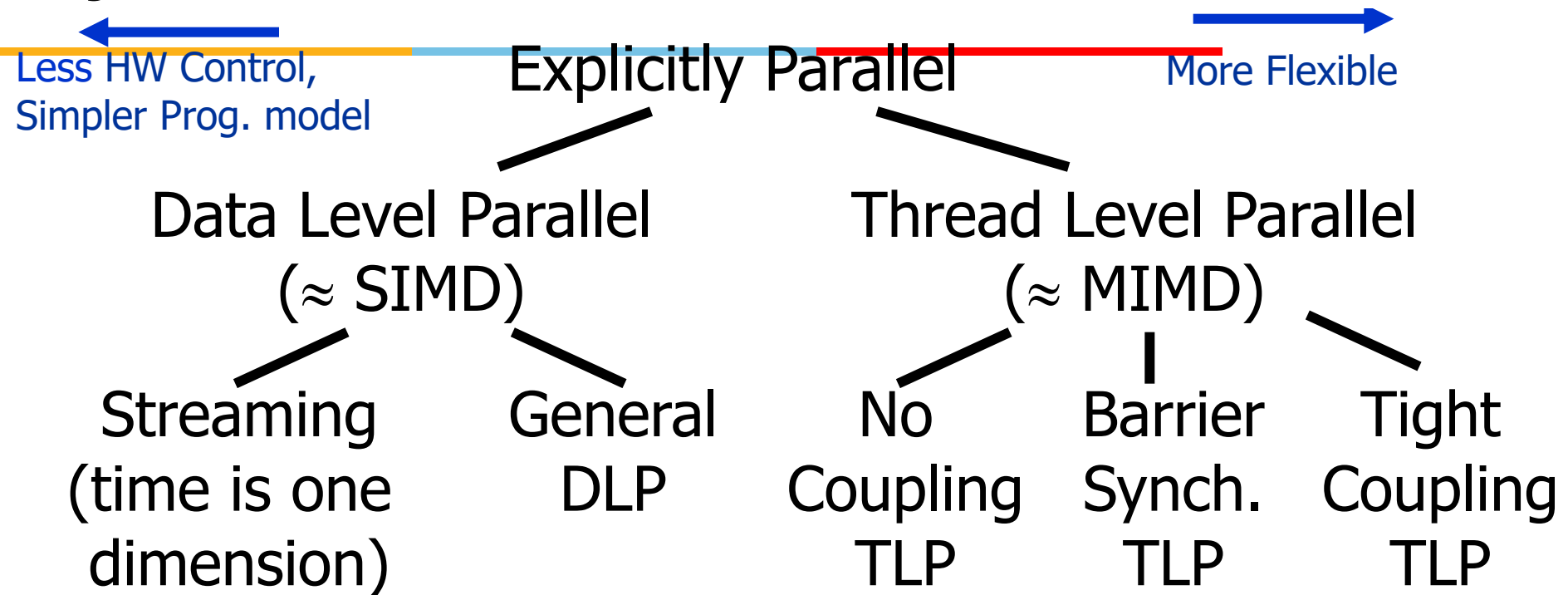
Loop Level

- recurrences

Procedure level

Program level

Style of Parallelism



Programmer wants code to run on as many parallel architectures as possible so (if possible)

Architect wants to run as many different types of parallel programs as possible so

- Development is very slow
- ISAs varied
- No. of operands
- Implied operands
- Operands may be stored in stack

Instruction types and opcodes



- Four classes of instructions:
- **integer arithmetic/logic instructions:** opcodes for two's complement operations are ADD, SUB, MULT, and their unsigned equivalents are ADDU, SUBU, and MULTU
- **floating-point arithmetic instructions:** The purpose is to widen the range of representable numbers and to avoid arithmetic overflows and underflows. FADD, FMUL, FDIV.
- **memory transfer instructions:** LB, LH, LW, and LD; SB, SH, SW, and SD
- **control instructions.**

Instruction mixes



- a distribution of instructions is called an *instruction mix*.
- Two kinds of instruction mixes: static or dynamic.
- A static instruction mix is the distribution of instruction types in the program code.
- A dynamic instruction mix is the distribution of instruction types that are
- executed.
- In a dynamic mix one instruction at a given PC may be executed and counted multiple times (if it is part of a loop or a subroutine), whereas it is counted only once in the static mix.
- Static mixes are used to size instruction caches and memories.
- Dynamic mixes are much more helpful than static mixes as a primary guideline in designing the CPU; CPU design should focus on common case instructions in the dynamic instruction mix, i.e., instructions that consume the most time in the execution of a typical program.

Instruction operands



- Operands inside the CPU:
 1. Accumulator: an implicit operand inside the CPU
 2. Evaluation Stack: Operands are in a hardware structure accessed as a stack.
 3. Registers: managed by the compiler.
 4. Immediate operands: constants stored in the instruction.
- Memory operands:
 - Memory operands are pointed to by their *effective address*.
 - Addressing modes
 - Number of memory operands

Addressing modes



Table 3.2 Examples of addressing modes in a register-based machine; (R_i) denotes “the value in R_i ”; $MEM[x]$ refers to the content of memory location at address x

Mode	Example	Meaning
Register-direct	ADD $R1, R2, R3$	$R1 \leftarrow (R2) + (R3)$
Immediate	ADDI $R1, R2, \#15$	$R1 \leftarrow (R2) + 15$
Displacement	LOAD $R1, \#20(R2)$	$R1 \leftarrow MEM[(R2) + 20]$
Register-indirect	LOAD $R1, (R2)$	$R1 \leftarrow MEM[(R2)]$
Memory-absolute	LOAD $R1, \#2000$	$R1 \leftarrow MEM[2000]$
Memory-indirect	LOAD $R1, @(R2)$	$R1 \leftarrow MEM[MEM[(R2)]]$
Post-increment	LOAD $R1, (R2) +$	$R1 \leftarrow MEM[(R2)]$ then $R2 \leftarrow (R2) + \text{size}$
Pre-decrement	LOAD $R1, -(R2)$	$R2 \leftarrow (R2) - \text{size}$ then $R1 \leftarrow MEM[(R2)]$
PC-relative	BEZ $R1, \#166$	$PC \leftarrow (PC) + 4 + 166$

Exceptions, traps, and interrupts



- Exceptions (as the word indicates) are very rare events that are triggered by the hardware and force the processor to execute an exception handler, caused by instruction, external interrupts, or hardware malfunctions.
- Interrupts:
 - **I/O device interrupts**
 - **Operating system calls**
 - **Instruction tracing and breakpoints**
 - **Integer or floating-point exceptions**
 - **Page faults**
 - **Undefined instruction**
 - **Hardware failure/alarm**
 - **Power failures**

CISC vs RISC



- **Characteristics of RISC –**
- Simpler instruction, hence simple instruction decoding.
- Instruction comes undersize of one word.
- Instruction takes a single clock cycle to get executed.
- More general-purpose registers.
- Simple Addressing Modes.
- Fewer Data types.
- A pipeline can be achieved.

Characteristics of CISC –

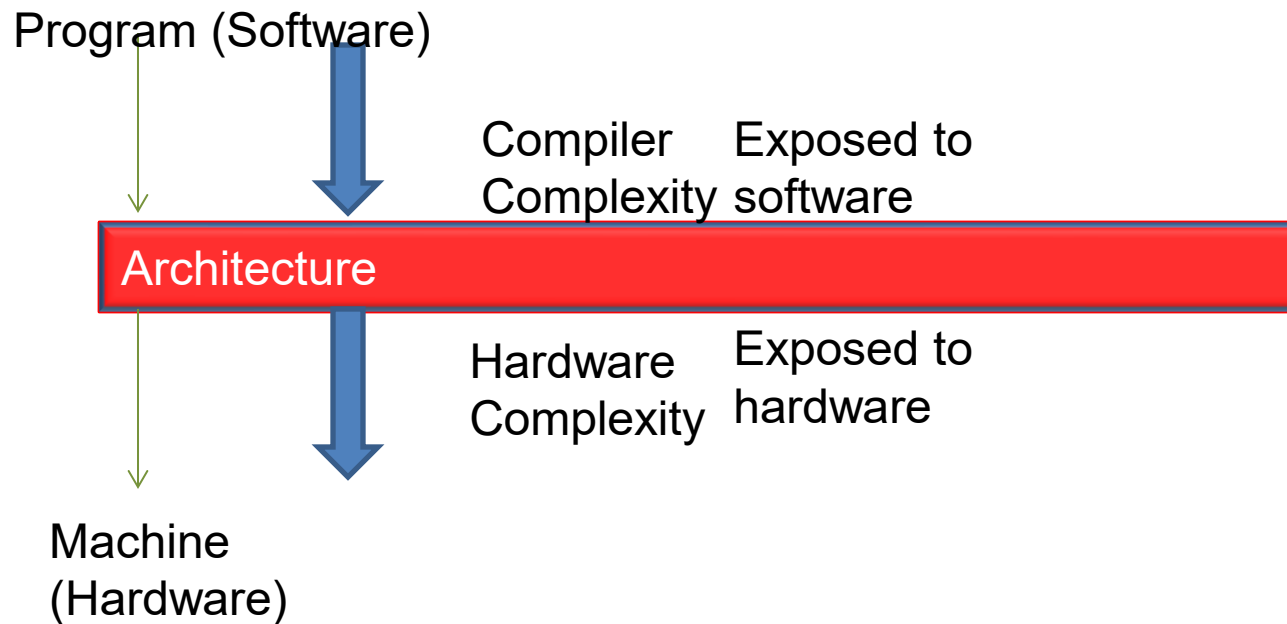
- Complex instruction, hence complex instruction decoding.
- Instructions are larger than one-word size.
- Instruction may take more than a single clock cycle to get executed.
- Less number of general-purpose registers as operations get performed in memory itself.
- Complex Addressing Modes.
- More Data types.

Dynamic – Static interface



- Separates what is done
- Compile Time
 - Statically
- At run time
 - Dynamically

Dynamic – Static interface



Directed executable language

